

Техническое задание

Sejire — полностью децентрализованный протокол вечного хранения генеалогического дерева

Назначение документа

Этот документ фиксирует техническое задание на создание Sejire как полностью децентрализованного протокола, в котором **интерфейс, логика, состояние, опубликованные версии и история сообщений** живут только в экосистеме Arweave + АО. Arweave поддерживает модель permanent information storage и permaweb-приложений, а АО предоставляет actor-oriented процессы, взаимодействующие через сообщения и воспроизводимое состояние

Документ написан так, чтобы быть понятным разработчику, техническому архитектору и языковой модели, которая будет использовать его как инструкцию для проектирования и реализации. Внутри намеренно подробно описаны не только функции, но и архитектурная логика, ограничения и порядок принятия решений, чтобы не возникло ошибочного возврата к централизованной модели

Цель проекта

Sejire должен стать протоколом публикации, просмотра и версионирования генеалогических деревьев, где опубликованные данные невозможно незаметно изменить или удалить, а каждая новая редакция дерева фиксируется как новая версия в Arweave с логическим управлением через АО-процесс.

Смысл системы состоит не в хранении обычных пользовательских черновиков на централизованном сервере, а в создании протокольной схемы, где:

- интерфейс приложения размещен в Arweave как permaweb app;
- каждая опубликованная версия дерева хранится в Arweave как immutable payload;

- АО-процесс управляет логикой дерева, head-версией, историей публикаций и правилами допустимых переходов состояния;
- никакой централизованный backend, локальный сервер, database.json или приватный REST API не участвуют в модели истины.

Базовые принципы

1. Полная децентрализация

Все компоненты должны находиться только в Arweave и АО. Если компонент нужен для существования опубликованного дерева, истории версий или интерфейса взаимодействия, он не должен зависеть от Render, VPS, private API, Postgres, MongoDB, local server или любого другого централизованного слоя.

2. Неизменяемость опубликованного

После публикации конкретной версии дерева данные не редактируются. Любое изменение создается как новая версия с явной ссылкой на предыдущую. Это согласуется с immutable version chain-подходом и природой Arweave как append-only хранилища.

3. АО управляет логикой, Arweave хранит данные

АО не должен быть использован как место хранения сырого UI-state или случайных временных объектов. АО должен хранить и вычислять протокольное состояние: кто владелец дерева, какая head-версия актуальна, какие версии зарегистрированы, какой payload является каноническим и как связаны версии между собой.

Arweave должен хранить:

- опубликованный frontend;
- модули процессов АО;
- payload каждой опубликованной версии дерева;
- служебные manifests и метаданные, если они нужны для резолва версии.

4. Источник истины только один

Source of truth не может находиться одновременно в АО, браузере, JSON-файле и каком-либо сервере. Истина определяется опубликованным payload в Arweave и состоянием АО-процесса, воспроизводимым из message log.

Предметная область

Sejire хранит не произвольный визуальный макет дерева, а **каноническое генеалогическое содержание**: персоналии, отношения между ними, метаданные версии и ссылки на источник публикации. Отображение в виде вертикального дерева, bow-tie, шежіре, PDF-раскладки или SVG — это клиентские представления, а не канонический формат хранения.

Поэтому в протокол должны попадать факты и структура, а не детали интерфейса. Например, положение карточки на холсте, zoom, координаты drag-and-drop, открытые модальные окна и выбранный режим печати не являются частью вечной генеалогической записи и не должны составлять Published Payload.

Состав системы

Компонент	Расположение	Назначение
SEJIRE Web App	Arweave / permaweb	Децентрализованный web-интерфейс для создания и просмотра дерева.
AO Tree Process	АО	Логика одного дерева: head, версии, публикация, чтение состояния.

AO Registry Process	AO	Глобальный индекс деревьев, резолв дерева по идентификатору и владельцу.
Version Payload	Arweave	Immutable JSON одной опубликованной версии дерева
Version Manifest	Arweave	Метаданные версии, ссылки на payload и parentVersionId.
AO Module Code	Arweave	Код процесса, доступный для запуска через spawn.

То, чего в проекте быть не должно

Следующие компоненты запрещены в целевой архитектуре:

- централизованный backend на Go, Node, Python и т.д.;
- локальный файл database.json как база истины;
- REST API вида /api/persons, /api/snapshot, /api/import;
- фейковые Arweave TX ID, полученные из локального SHA-256 вместо реальной публикации;
- смешение канонических данных и UI-представления дерева в одном объекте.

Архитектурная модель

Общая модель работы

Система должна работать по следующей схеме:

1. Пользователь открывает permaweb-приложение по URL, опубликованному в Arweave.techcrunch+1

2. Приложение получает из URL идентификатор дерева или processId, если дерево уже существует.
3. Если дерева нет, приложение инициирует создание AO Tree Process или запись в Registry Process.
4. Пользователь создает или редактирует дерево в интерфейсе.
5. При публикации приложение формирует канонический payload версии.
6. Payload публикуется в Arweave.
7. Затем AO Tree Process получает сообщение PublishVersion, валидирует его и обновляет head-версию дерева.
8. В дальнейшем опубликованное дерево открывается по ссылке как read-only состояние, а новая правка публикуется как следующая версия.

9.

Важное архитектурное различие

Ссылка не должна открывать “голый АО-процесс”. АО-процесс не является HTML-страницей. Ссылка должна открывать frontend, опубликованный на Arweave, а frontend должен уметь работать с АО-processId. Это обязательная часть архитектуры и её нельзя подменять абстракцией “процесс по URL”.

Идентификация сущностей

В системе должны существовать следующие идентификаторы:

- treeld — стабильный идентификатор дерева;
- processId — идентификатор AO Tree Process;
- versionId — идентификатор конкретной опубликованной версии;
- parentVersionId — ссылка на предыдущую версию;
- payloadTxId — Arweave transaction id полезной нагрузки версии;
- manifestTxId — Arweave transaction id манифеста версии;
- owner — идентификатор владельца/публикатора дерева.

Нужно обеспечить, чтобы treeld и versionId были детерминированными или как минимум криптографически надежными и не зависели от слабого генератора на основе time.Now() и случайного четырехзначного числа

Модель данных

Каноническая сущность Person

Каждая персоналия должна храниться как объект, содержащий только устойчивые предметные поля. Минимально допустимая схема:

```
json
{
  "personId": "pers_...",
  "fullName": "...",
  "sex": "M|F|U",
  "birth": {
    "date": "optional string",
    "place": "optional string"
  },
  "death": {
    "date": "optional string",
    "place": "optional string"
  },
  "notes": "optional string",
  "sources": []
}
```

В этой схеме нельзя хранить поля, отражающие только конкретное визуальное представление, например `x`, `y`, `canvasOrder`, `roleInCurrentLayout`, `isExpanded`, `selected` и подобные свойства, если они не являются частью канонического факта о родстве.

Каноническая сущность Relationship

Текущая модель `TargetID + Role` должна быть заменена на явные отношения. Минимальная форма:

```
json
{
  "relationshipId": "rel_...",
  "fromPersonId": "pers_...",
  "toPersonId": "pers_...",
}
```

```
"type": "parentOf|childOf|spouseOf|adoptiveParentOf|other",
"meta": {}
}
```

Это критически важно, потому что строки вроде Отец, Мать, Прадед и 5-е колено являются UI/генерационными ярлыками, а не универсальной моделью родства. Для вечного протокола нужна нормализованная графовая схема, пригодная для воспроизводимого расчета разных визуальных представлений.

Version Payload

Каждая опубликованная версия дерева должна храниться как immutable JSON payload в Arweave. Рекомендуемая структура:

```
json
{
  "schemaVersion": "1.0.0",
  "treeId": "tree_...",
  "versionId": "ver_...",
  "parentVersionId": "ver_... or null",
  "createdAt": "ISO-8601",
  "createdBy": "wallet or owner id",
  "persons": [],
  "relationships": [],
  "metadata": {
    "title": "optional",
    "description": "optional",
    "culture": "optional",
    "language": "optional"
  }
}
```

Этот payload должен быть **канонизирован** перед хешированием и публикацией. Один и тот же набор данных должен давать одинаковый хеш. Это нужно для проверки целостности, предсказуемости версий и корректного сравнения версий между собой.

Version Manifest

Поверх payload нужно хранить manifest-объект, который является связующим слоем между АО и Arweave:

```
json
{
  "schemaVersion": "1.0.0",
  "treeld": "tree_...",
  "versionId": "ver_...",
  "parentVersionId": "ver_... or null",
  "payloadTxId": "...",
  "payloadHash": "...",
  "createdAt": "ISO-8601",
  "owner": "...",
  "processId": "..."
}
```

Manifest должен публиковаться в Arweave и именно он должен фигурировать в сообщениях АО-процессу при публикации новой версии.

АО-процессы

1. Registry Process

Registry Process нужен для:

- регистрации новых деревьев;
- поиска дерева по treeld;
- поиска дерева по owner;
- хранения соответствия treeld -> processId;
- хранения headVersionId или ссылки на Tree Process, который сам знает свой head.

Состояние Registry Process

```
json
{
  "schemaVersion": "1.0.0",
```



```

"trees": {
  "tree_123": {
    "processId": "...",
    "owner": "...",
    "createdAt": "...",
    "headVersionId": "..."
  }
}

```

Сообщения Registry Process

- CreateTree
- ResolveTree
- ListTreesByOwner
- UpdateHead
- GetRegistryEntry

2. Tree Process

Tree Process — основной протокольный объект. Один процесс соответствует одному дереву или одной линии версий конкретного дерева.

Состояние Tree Process

json

```

{
  "schemaVersion": "1.0.0",
  "treeId": "tree_...",
  "owner": "...",
  "createdAt": "...",
  "headVersionId": "ver_...",
  "versions": [
    {
      "versionId": "ver_...",
      "parentVersionId": null,
      "manifestTxId": "...",
      "payloadTxId": "...",

```

```
"payloadHash": "...",  
"createdAt": "...",  
"createdBy": "..."  
}  
],  
"status": "active"  
}
```

Сообщения Tree Process

Минимально обязательные сообщения:

- InitTree
- PublishVersion
- GetHead
- GetVersionMeta
- ListVersions
- GetTreeState

Логика PublishVersion

При получении PublishVersion процесс обязан:

1. проверить treeld;
2. проверить, что отправитель сообщения имеет право публиковать новую версию;
3. проверить корректность parentVersionId;
4. проверить, что payloadTxId и payloadHash присутствуют;
5. проверить, что версия не дублирует уже зарегистрированную;
6. добавить новую запись в versions;
7. обновить headVersionId;
8. при необходимости уведомить Registry Process о смене головы.

Публикация сайта

Frontend приложения должен быть собран как статический SPA и опубликован в Arweave/permaweb. Это не дополнительная опция, а обязательное требование, иначе сайт остается централизованной точкой отказа.

Frontend должен:

- открываться по gateway/permaweb URL;
- уметь читать treeld или processId из маршрута;
- использовать AO connect SDK для общения с процессом;
- быть полностью работоспособным без обращения к централизованному API.

Frontend-маршруты

Рекомендуется следующая маршрутизация:

Маршрут	Назначение
/	Главная страница протокола
/new	Создание нового дерева
/tree/:processId	Просмотр или редактирование дерева по процессу
/tree/:processId/history	История версий
/tree/:processId/version/:versionId	Просмотр конкретной версии

Такой URL-дизайн не противоречит децентрализации, потому что сам frontend также размещен в Arweave, а маршруты обрабатываются клиентом.

UI/UX-требования

Интерфейс должен повторять смысл текущего локального сайта как визуального референса, но не его централизованную реализацию.

Текущее приложение показывает, как пользователь должен воспринимать продукт: дерево, карточки персон, форма добавления, SVG-связи, экспорт и печать. Этот UX можно использовать как ориентир при создании нового permaweb frontend.

При этом обязательны следующие режимы:

- режим создания дерева;
- режим просмотра опубликованной head-версии;
- режим просмотра истории версий;
- режим публикации новой версии;
- режим подтверждения перед отправкой в Arweave.

Режимы работы интерфейса

Create Mode

Пользователь вводит данные, формирует дерево и видит его визуально.

Review Before Publish Mode

Перед публикацией интерфейс обязан показать:

- что именно будет опубликовано;
- размер payload;
- parentVersionId, если это не первая версия;
- предупреждение о неизменяемости публикации;
- стоимость публикации, если этот механизм реализован на клиенте.

Published View Mode

После публикации версия открывается как read-only. Разрешен просмотр, экспорт и печать, но не скрытое редактирование опубликованного содержимого.

New Version Mode

Если пользователь хочет изменить дерево, интерфейс должен создавать новую рабочую копию на основе head-версии и при

публикации оформлять её как новую версию с указанием parentVersionId.

Требования к канонизации данных

Перед публикацией нужно выполнить канонизацию:

- отсортировать массивы persons и relationships детерминированно;
- нормализовать пустые значения;
- исключить временные поля интерфейса;
- нормализовать строки и даты;
- сериализовать JSON в стабильном порядке ключей;
- вычислить payloadHash только после полной канонизации.

Без канонизации невозможно корректно сравнивать версии, проверять неизменность и валидировать соответствие payload и manifest.

Требования к безопасности

Подпись действий

Все критические действия — создание дерева, публикация версии, смена владельца, если она вообще допускается, — должны быть подписаны. Процесс не должен доверять данным только потому, что их прислал какой-то frontend.

Валидация входящих сообщений

АО-процесс обязан валидировать:

- наличие всех обязательных полей;
- типы данных;
- допустимость перехода состояния;
- соответствие owner / actor правилам доступа;
- отсутствие дубликатов и повторной публикации уже существующей версии.

Защита от неконсистентных данных

Процесс обязан отвергать:

- публикации с неизвестным parentVersionId;

- циклические ссылки версий;
- пустые payload без персон и без связей, если бизнес-правила этого не допускают;
- payload, где relationship указывает на несуществующего человека.

Формальная бизнес-логика версий

Система должна работать по следующим правилам:

1. Первая опубликованная версия дерева имеет `parentVersionId = null`.
2. Каждая следующая версия должна ссылаться на предыдущую актуальную head-версию
3. Head-версия — это последняя валидно опубликованная версия дерева, признанная процессом.
4. Старые версии не удаляются и не перезаписываются.
5. Просмотр по умолчанию должен открывать head-версию.

Ограничение первой версии протокола

Чтобы не сорвать MVP, первая версия Sejire должна отказаться от лишней сложности. В v1 не нужно внедрять:

- мультивладельцев;
- сложные права доступа;
- совместное редактирование в реальном времени;
- приватное шифрование отдельных полей;
- ZK-механику;
- экономику подписки внутри самого протокола;
- социальные функции, комментарии и публичные профили.

MVP должен решать одну задачу: создать дерево, опубликовать immutable-версию в Arweave и получить доступ к ней через AO process и permaweb UI.

Нефункциональные требования

Производительность

Frontend должен быстро открывать дерево по processId и уметь лениво подгружать историю версий. Не допускается загрузка тяжелого UI-state из централизованного API.

Надежность

Даже если один gateway недоступен, приложение должно оставаться архитектурно переносимым между gateway и AO nodes, поскольку frontend и данные находятся в Arweave/AO, а не на приватном сервере.

Переносимость

Все схемы, сообщения и структуры данных должны быть документированы так, чтобы другой разработчик мог написать альтернативный клиент без доступа к исходному frontend.

Этапы реализации

Этап 1. Protocol Design

Нужно зафиксировать:

- JSON Schema для Person;
- JSON Schema для Relationship;
- JSON Schema для Version Payload;
- JSON Schema для Version Manifest;
- список АО-сообщений;
- state-модель Registry и Tree Process.

Этап 2. AO Module Development

Нужно реализовать:

- код Registry Process;
- код Tree Process;
- обработчики CreateTree, PublishVersion, GetHead, ListVersions;
- валидацию сообщений;
- обновление head-версии.

Этап 3. Permaweb Frontend

Нужно реализовать:

- новый frontend без Go backend;
- визуальный редактор дерева;
- канонизацию payload;
- публикацию payload и manifest в Arweave;
- интеграцию с aoconnect для вызова AO process

Этап 4. Integration

Нужно связать:

- создание дерева -> spawn Tree Process;
- публикацию payload -> запись manifest -> PublishVersion;
- чтение head -> отображение дерева;
- чтение списка версий -> экран истории [.cookbook_ao.arweave+2](#)

Этап 5. Permaweb Deployment

Нужно опубликовать:

- frontend в Arweave;
- AO modules в Arweave;
- начальные manifests и служебные документы протокола при необходимости [.github+2](#)

Definition of Done

Проект считается реализованным только если одновременно выполнены все условия:

- frontend опубликован в Arweave и доступен через permaweb URL;
- создание и просмотр дерева работают без централизованного backend;
- Tree Process действительно существует в АО и отвечает на сообщения;
- публикация версии создает реальную запись в Arweave, а не локальный fake hash;
- АО хранит head-логику и историю версий;
- опубликованная версия дерева read-only и может быть просмотрена по ссылке;
- новая правка создается как новая версия, а не редактирование старой.

Критические ошибки, которых нельзя допустить

- Нельзя использовать обычный backend как скрытый костыль для хранения актуального состояния.
- Нельзя считать локальный JSON-файл или браузерный state источником истины после публикации.
- Нельзя хранить в payload данные, относящиеся только к визуальному layout дерева.
- Нельзя публиковать новую версию без явной связи с предыдущей.
- Нельзя оставлять fake snapshot-поведение под видом Arweave-интеграции.
- Нельзя смешивать AO process и permaweb page как один и тот же объект.

Итоговая формулировка задачи

Необходимо создать SEJIRE как полностью децентрализованный протокол и permaweb-приложение для вечного хранения генеалогических деревьев, где интерфейс размещен в Arweave, протокольная логика реализована через АО-процессы, а каждая опубликованная версия дерева фиксируется как immutable payload в Arweave и регистрируется в АО как новая head или историческая версия.

Необходимо реализовать систему так, чтобы любой разработчик или языковая модель, опираясь на этот документ, мог создать совместимый frontend, совместимые АО-процессы и совместимый формат хранения без обращения к централизованной серверной архитектуре.